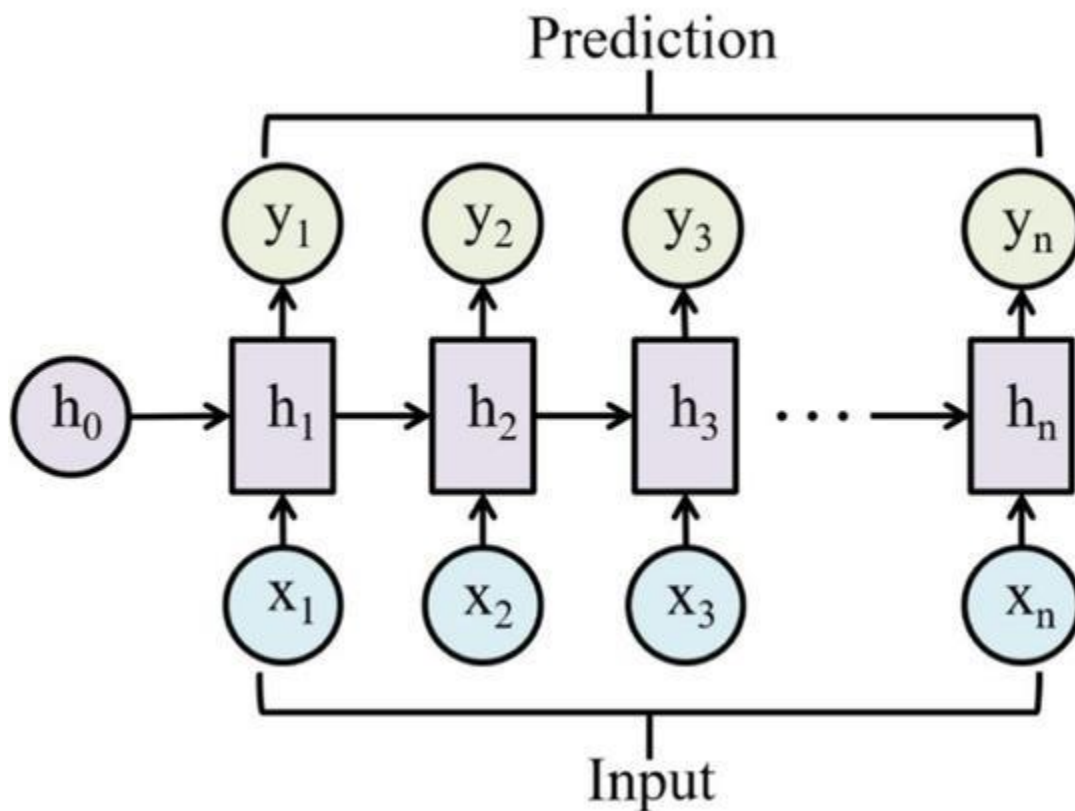


## Module 5

### Q. Sequence Learning Problem in RNN

=>



### Introduction

1. A sequence learning problem is a problem in which the order of data is important.
2. In sequence data, one value depends on previous values.
3. Traditional neural networks cannot remember previous inputs.
4. Recurrent Neural Networks (RNNs) are designed to handle sequence learning problems.
5. RNNs process one element at a time and remember past information using hidden states.

### What is Sequence Data

1. Sequence data is data arranged in a particular order.
2. The order of elements affects the meaning of the data.
3. Example: In a sentence, changing the order of words changes the meaning.
4. Example: In stock market data, future prices depend on previous prices.
5. Example: In speech signals, each sound depends on previous sounds.

## Why Traditional Neural Networks Fail

1. Traditional neural networks treat every input independently.
2. Traditional neural networks cannot store previous information.
3. Traditional neural networks cannot understand order in data.
4. Example: A normal neural network cannot understand the sentence “I am not happy” because the order of words matters.

## How RNN Solves Sequence Learning Problem

1. RNN processes data one step at a time.
2. RNN stores previous information in hidden states.
3. The hidden state acts like memory of previous inputs.
4. The output at each step depends on both current input and previous hidden state.
5. This allows RNN to learn relationships across time steps.

## Mathematical Representation

1. The hidden state is calculated as
2. The symbol ( $h_t$ ) represents the current hidden state.
3. The symbol ( $h_{t-1}$ ) represents the previous hidden state.
4. The symbol ( $x_t$ ) represents the current input.
5. The function ( $f$ ) is usually Tanh or ReLU.

## Types of Sequence Learning Problems

### 1. One-to-One

1. One input gives one output.
2. Example: Image classification.

### 2. One-to-Many

1. One input gives multiple outputs.
2. Example: Image caption generation.

### 3. Many-to-One

1. Multiple inputs give one output.
2. Example: Sentiment analysis of a sentence.

### 4. Many-to-Many

1. Multiple inputs give multiple outputs.
2. Example: Language translation.

## Q. Comparision between LSTM and GRU

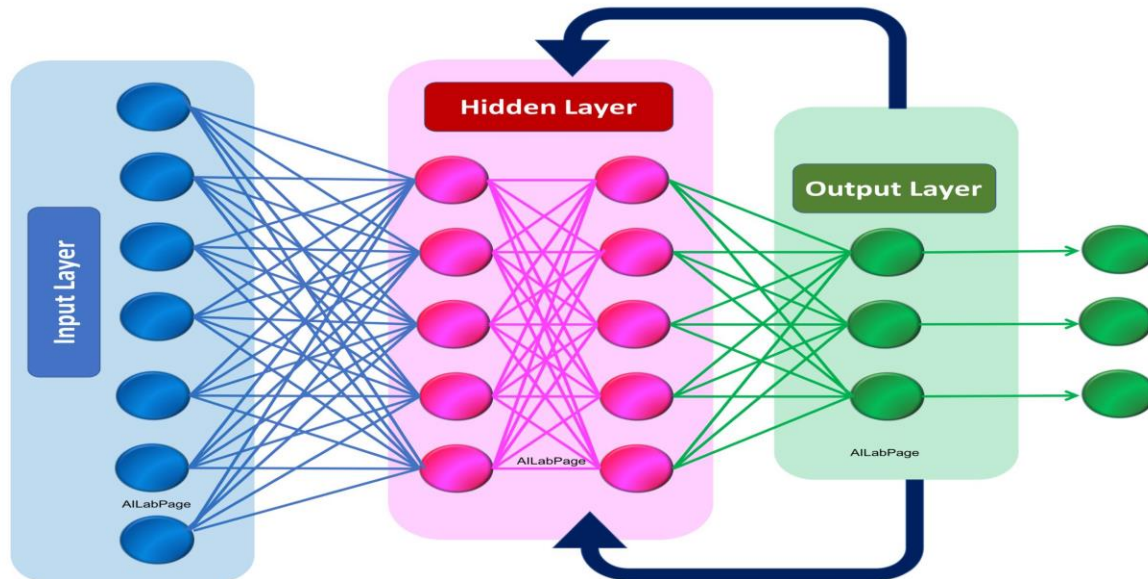
=>

| Feature                   | LSTM (Long Short-Term Memory)                            | GRU (Gated Recurrent Unit)                                   |
|---------------------------|----------------------------------------------------------|--------------------------------------------------------------|
| Full Form                 | Long Short-Term Memory                                   | Gated Recurrent Unit                                         |
| Introduced By             | Introduced by Hochreiter and Schmidhuber in 1997         | Introduced by Cho et al. in 2014                             |
| Main Purpose              | Solves long-term dependency problem in RNN               | Solves long-term dependency problem in RNN                   |
| Number of Gates           | Uses 3 gates                                             | Uses 2 gates                                                 |
| Gates Used                | Input gate, Forget gate, Output gate                     | Update gate and Reset gate                                   |
| Memory Cell               | Has a separate memory cell                               | Does not have a separate memory cell                         |
| Hidden State              | Hidden state and cell state are separate                 | Hidden state and memory are combined                         |
| Complexity                | More complex architecture                                | Simpler architecture                                         |
| Number of Parameters      | More parameters                                          | Fewer parameters                                             |
| Training Speed            | Slower training                                          | Faster training                                              |
| Computation Cost          | Higher computational cost                                | Lower computational cost                                     |
| Memory Usage              | Requires more memory                                     | Requires less memory                                         |
| Performance on Small Data | May overfit on small datasets                            | Often performs better on small datasets                      |
| Long Sequence Handling    | Better for very long sequences                           | Good for long sequences but slightly less powerful than LSTM |
| Ease of Implementation    | More difficult to implement                              | Easier to implement                                          |
| Accuracy                  | Can provide better accuracy for complex tasks            | Gives similar accuracy with less computation                 |
| Popular Applications      | Machine translation, speech recognition, text generation | Chatbots, sentiment analysis, time series prediction         |

## Q. Recurrent Neural Network (RNN) Architecture

=>

# Recurrent Neural Networks



## Introduction

1. Recurrent Neural Network (RNN) is a type of neural network designed for sequence data.
2. RNN is mainly used when the current input depends on previous inputs.
3. RNN has a memory mechanism that stores information from previous time steps.
4. RNN is widely used in speech recognition, language translation, and text generation.

## Basic Architecture of RNN

1. The RNN architecture contains an input layer, hidden layer, recurrent connection, and output layer.
2. The same hidden layer is repeated for every time step.
3. The hidden layer receives both current input and previous hidden state.
4. The hidden state acts like memory of the network.
5. The output depends on both current input and past information.

## Components of RNN Architecture

### 1. Input Layer

1. The input layer receives sequence data one element at a time.
2. The input at time step ( $t$ ) is represented as  $(x_t)$ .
3. Example: In a sentence, each word is given one by one to the RNN.

## 2. Hidden Layer

1. The hidden layer processes the current input and previous hidden state.
2. The hidden state at time (t) is represented as  $(h_t)$ .
3. The hidden state stores information from previous inputs.
4. The function (f) is usually Tanh or ReLU.

## 3. Recurrent Connection

1. The recurrent connection sends information from one hidden state to the next hidden state.
2. This connection helps the network remember previous information.
3. The recurrent connection is the main difference between RNN and normal neural networks.

## 4. Output Layer

1. The output layer produces the prediction at each time step.
2. The output at time (t) is represented as  $(y_t)$ .
3. Example: In language translation, each output may represent a translated word.

## Unfolded Representation of RNN

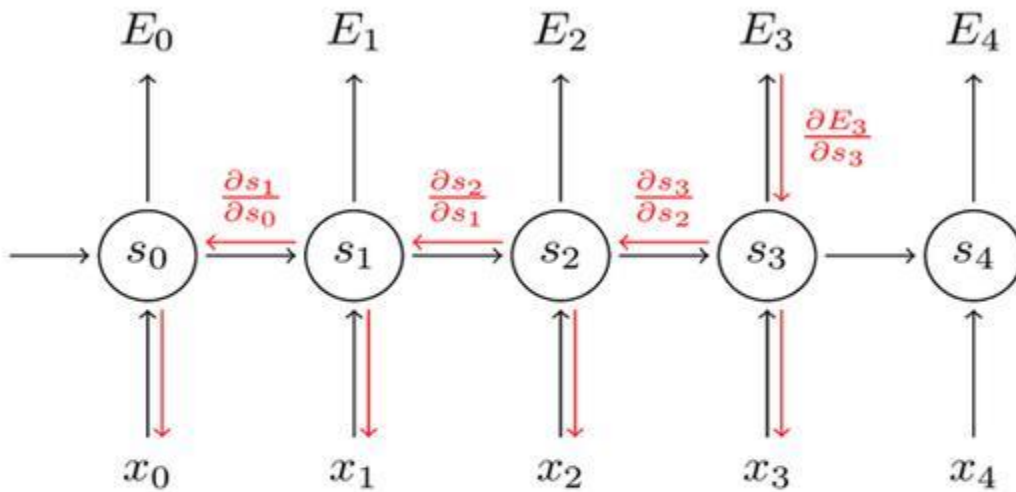
1. RNN can be represented as unfolded across multiple time steps.
2. Each time step has the same network structure and shared weights.
3. The hidden state moves from one step to the next step.
4. Example: For input sequence  $(x_1, x_2, x_3)$ , the hidden states become  $(h_1, h_2, h_3)$ .

## Working of RNN

1. The first input is given to the hidden layer.
2. The hidden layer produces an output and hidden state.
3. The hidden state is passed to the next time step.
4. The process continues until all sequence elements are processed.
5. The network uses previous information to make better predictions.

## Q. Backpropagation Through Time (BPTT) Updates in RNN

=>



# Backpropagation Through Time

## Introduction

1. Backpropagation Through Time (BPTT) is the training algorithm used for Recurrent Neural Networks (RNNs).
2. BPTT is an extension of the normal backpropagation algorithm.
3. BPTT is used because RNN contains repeated hidden layers across time steps.
4. BPTT updates the weights by considering errors from all time steps.

## Need for BPTT in RNN

1. In RNN, the same weights are used at every time step.
2. The output at the current step depends on previous hidden states.
3. Therefore, the error must be propagated backward through all previous time steps.
4. Normal backpropagation cannot handle time dependencies in sequence data.

## Working of BPTT

1. The RNN is first unfolded across all time steps.
2. Each unfolded step behaves like a separate neural network layer.
3. Forward propagation calculates outputs and hidden states for all time steps.

4. The total loss is calculated by adding losses from all outputs.
5. The error is then propagated backward from the last time step to the first time step.
6. Gradients are calculated for each weight at every time step.
7. The gradients are added together because the same weights are shared across all time steps.
8. The weights are updated using the total gradient.

### **Steps of BPTT Update**

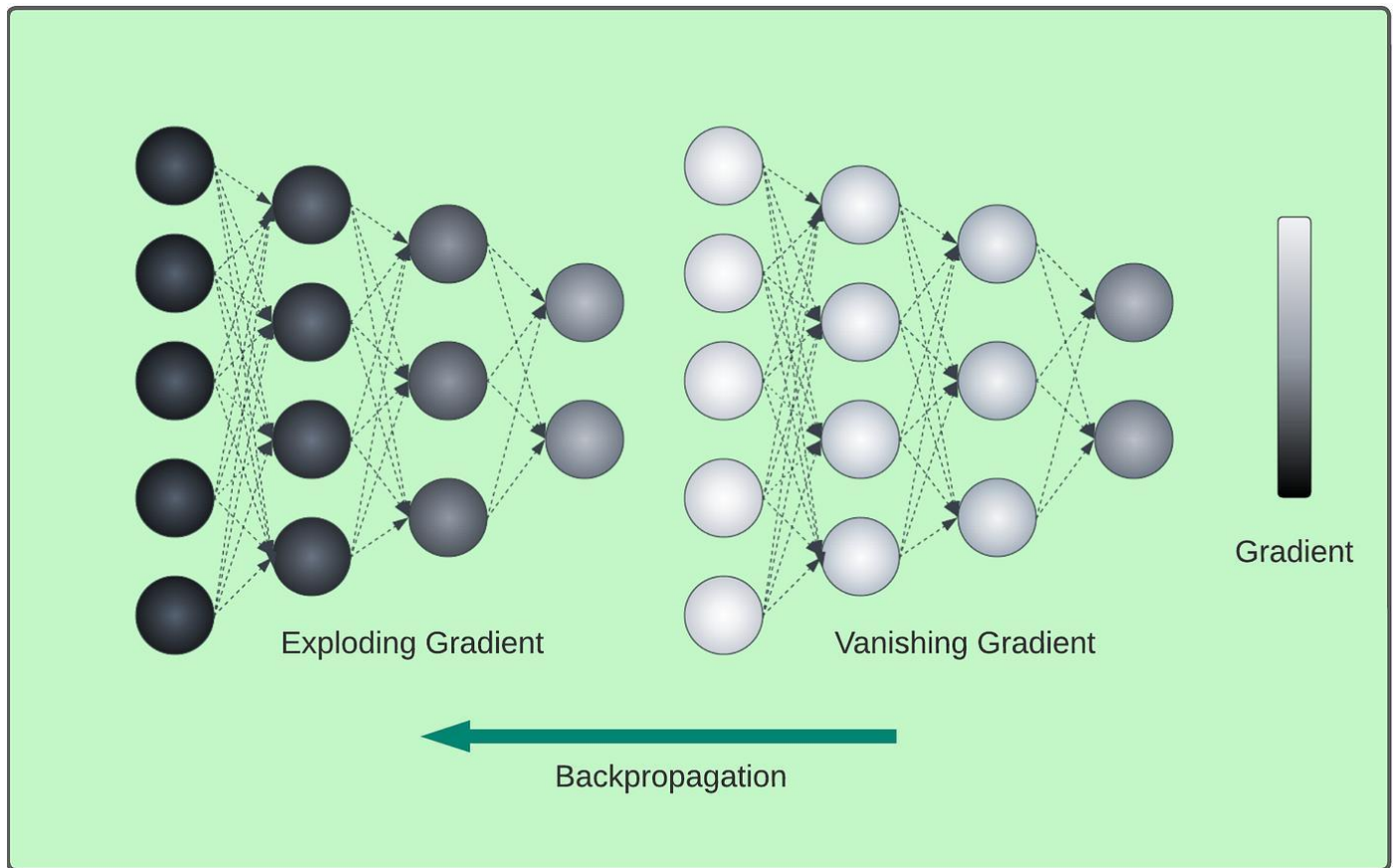
1. Input sequence is passed through the RNN.
2. Hidden states and outputs are calculated for each time step.
3. The total loss is computed.
4. Error is propagated backward from the final time step to earlier steps.
5. Gradients for recurrent weights, input weights, and output weights are computed.
6. The gradients from all time steps are summed.
7. The weights are updated using gradient descent.

### **Problems in BPTT**

1. BPTT suffers from vanishing gradient problem.
2. Very small gradients make learning difficult for long sequences.
3. BPTT also suffers from exploding gradient problem.
4. Very large gradients make training unstable.
5. LSTM and GRU are used to reduce these problems.

## Q. Vanishing and Exploding Gradients

=>



### Introduction

1. Vanishing gradient and exploding gradient are common problems in deep neural networks and RNNs.
2. These problems occur during backpropagation when gradients become very small or very large.
3. Gradients are used to update weights during training.
4. If gradients are not proper, the network cannot learn effectively.

### Vanishing Gradient Problem

1. Vanishing gradient happens when gradient values become very small.
2. Small gradients make the weight updates almost zero.
3. Early layers of the network learn very slowly.
4. The network may stop learning completely.
5. Vanishing gradient is very common in deep networks and RNNs.
6. Sigmoid and Tanh activation functions often cause vanishing gradients.
7. In RNN, vanishing gradient makes it difficult to remember old information from long sequences.
8. Example: In a long sentence, the network may forget words at the beginning.

## Causes of Vanishing Gradient

1. Repeated multiplication of values less than 1 makes gradients very small.
2. Deep networks contain many layers, so gradients shrink as they move backward.
3. Sigmoid activation produces derivatives between 0 and 1, which reduces gradients further.

## Solutions for Vanishing Gradient

1. ReLU activation function helps reduce vanishing gradient.
2. LSTM and GRU are designed to handle vanishing gradients in RNNs.
3. Proper weight initialization helps reduce this problem.
4. Batch normalization also helps improve gradient flow.

## Exploding Gradient Problem

1. Exploding gradient happens when gradient values become very large.
2. Large gradients make weight updates extremely high.
3. The network becomes unstable during training.
4. The loss value may increase instead of decreasing.
5. Exploding gradient is also common in deep networks and RNNs.
6. Example: The model may produce NaN values during training due to very large gradients.

## Causes of Exploding Gradient

1. Repeated multiplication of values greater than 1 makes gradients very large.
2. Large weights and high learning rates increase the chance of exploding gradients.
3. Very deep networks are more likely to face this problem.

## Solutions for Exploding Gradient

1. Gradient clipping is commonly used to control very large gradients.
2. Lower learning rates help reduce exploding gradients.
3. Proper weight initialization helps avoid unstable training.
4. Using LSTM and GRU can also help reduce this problem in RNNs.

## Difference Between Vanishing and Exploding Gradient

| Feature            | Vanishing Gradient                            | Exploding Gradient                               |
|--------------------|-----------------------------------------------|--------------------------------------------------|
| Gradient Size      | Very small                                    | Very large                                       |
| Effect on Training | Learning becomes slow                         | Training becomes unstable                        |
| Weight Update      | Very small updates                            | Very large updates                               |
| Common Cause       | Repeated multiplication by values less than 1 | Repeated multiplication by values greater than 1 |
| Main Problem       | Model forgets information                     | Model diverges during training                   |

|          |                                      |                                          |
|----------|--------------------------------------|------------------------------------------|
| Solution | ReLU, LSTM, GRU, batch normalization | Gradient clipping, smaller learning rate |
|----------|--------------------------------------|------------------------------------------|